

Evolução da API, CRUD e Arquitetura

Linguagem de Programação III

Problema real do código anterior

- Crescimento rápido de if/else
- Código duplicado
- Difícil manutenção
- Baixa escalabilidade
- Dificuldade de leitura
- Estrutura não profissional

Exemplo de código problemático

- `if (req.url === '/usuarios') {}`
- `else if (req.url === '/usuarios/1') {}`
- `else if (req.url === '/usuarios/2') {}`

Pergunta central

- E se existirem 100 usuários?
- Vamos criar 100 ifs?
- Isso é sustentável?
- Como sistemas reais fazem isso?
- Como evitar repetição?
- Como generalizar?

Ideia da solução

- Criar rotas genéricas/dinâmicas:
 - Extrair dados da URL
 - Separar responsabilidades
 - Reutilizar código
 - Evitar repetição
 - Organizar a aplicação

O que são rotas dinâmicas

- Rotas que aceitam parâmetros
- Exemplo: /usuarios/1
- O número muda, a rota é a mesma
- Mais flexível
- Menos código
- Mais próximo do mundo real

Detectando rota dinâmica

```
if (req.url.startsWith('/usuarios/')) {  
    // tratar dinamicamente  
}
```

Por que startsWith?

- Permite identificar prefixos
- Evita múltiplos ifs
- Funciona com qualquer ID
- Simples e eficiente
- A base para frameworks

Extraindo o ID da URL

- Precisamos capturar o número
- Usamos `split('/')`
- Transformamos string em array
- Identificamos posição do ID
- Convertemos para número
- Usamos no sistema

Código de extração do ID

```
const partes = req.url.split('/');  
const id = parseInt(partes[2]);
```

Como o split funciona

- Divide a string pelo separador
- Exemplo: /usuarios/1
- Resultado: ['', 'usuarios', '1']
- ID está na posição 2
- Sempre verificar índice
- Evitar erros de posição

Importância do parseInt

- Converte string em número
- Permite comparação correta
- Evita bugs silenciosos
- Obrigatório para find
- Boa prática
- Evita erros futuros

Servidor evoluído (versão 1)

```
import http from 'http';

const server = http.createServer((req, res) => {

  if (req.url === '/usuarios') {
    res.end('Lista de usuários');
  }

  else if (req.url.startsWith('/usuarios/')) {
    const partes = req.url.split('/');
    const id = parseInt(partes[2]);
    res.end('Usuário ' + id);
  }

});

server.listen(3000);
```

Limitação ainda existente

- Lógica ainda no servidor
- Código misturado
- Baixa organização
- Não segue arquitetura e precisa melhorar

Separação de responsabilidades

- Servidor → roteamento
- Controller → resposta
- Service → regras
- Repository → dados
- Cada parte com função clara
- Código profissional

Controller exemplo

```
export function buscarUsuarioController(id, res) {  
  
  const usuarios = [  
    { id: 1, nome: 'Ana' },  
    { id: 2, nome: 'Carlos' }  
  ];  
  
  const usuario = usuarios.find(u => u.id === id);  
  
  if (!usuario) {  
    res.statusCode = 404;  
    res.end('Usuário não encontrado');  
    return;  
  }  
  
  res.setHeader('Content-Type', 'application/json');  
  res.end(JSON.stringify(usuario));  
}
```


Servidor com controller

```
import http from 'http';
import { buscarUsuarioController } from './userController.js';

const server = http.createServer((req, res) => {

  if (req.url.startsWith('/usuarios/')) {

    const partes = req.url.split('/');
    const id = parseInt(partes[2]);

    buscarUsuarioController(id, res);

  } else {
    res.statusCode = 404;
    res.end('Rota não encontrada');
  }

});

server.listen(3000);
```

Fluxo completo da aplicação

- Cliente envia requisição
- Servidor identifica rota
- Controller processa
- Service aplica lógica
- Repository acessa dados
- Resposta retorna ao cliente

JSON – reforço essencial

- Formato padrão de APIs
- Baseado em JavaScript
- Leve e simples
- Compatível com frontend
- Necessita stringify
- Sempre usar corretamente

Tratamento de erro

- Usuário pode não existir
- Precisa validar resultado
- Retornar status 404
- Mensagem clara
- Evitar quebra do sistema

Erros comuns dos alunos

- Esquecer parseInt
- Erro no split
- Não tratar erro
- JSON incorreto
- Rota mal formada
- Misturar responsabilidades

CRUD + Arquitetura

O que é CRUD?

- CRUD é um acrônimo para operações básicas de dados
- Create → criar novos registros
- Read → consultar ou visualizar dados
- Update → alterar dados existentes
- Delete → remover dados do sistema
- Todo sistema que manipula dados utiliza CRUD

CRUD no mundo real

- Cadastro de usuários (Create)
- Listagem de produtos (Read)
- Atualização de perfil (Update)
- Exclusão de conta (Delete)
- Sistemas web são baseados nessas operações
- APIs existem principalmente para expor CRUD

Métodos HTTP (visão geral)

- GET → obter dados
- POST → criar novos dados
- PUT → atualizar dados existentes
- DELETE → remover dados
- Cada método representa uma ação específica
- Seguir padrão melhora organização da API

GET – Definição completa

- Usado para **recuperar dados do servidor**
- Não deve alterar o estado da aplicação
- Pode ser acessado via navegador
- Parâmetros são enviados pela URL
- É idempotente (não muda resultado)
- Mais utilizado em APIs

POST – Definição completa

- Usado para **criar novos recursos**
- Envia dados no corpo da requisição (body)
- Altera o estado do sistema
- Não é idempotente
- Muito usado em formulários
- Exige processamento no backend

PUT – Definição completa

- Usado para **atualizar recursos existentes**
- Substitui ou altera dados
- Requer identificação do recurso (ID)
- Também utiliza body
- Usado em edição de dados

DELETE – Definição completa

- Usado para **remover dados do sistema**
- Requer identificação do recurso
- Remove permanentemente ou logicamente
- Deve tratar erros (não encontrado)
- Importante para consistência do sistema
- Completa o ciclo CRUD

GET – Implementação prática

- Primeira operação do CRUD
- Retorna todos os usuários
- Não altera dados
- Resposta deve ser JSON
- Uso de Content-Type
- Uso de JSON.stringify

GET /usuarios

```
if (req.url === "/usuarios" && req.method === "GET") {  
  
    res.setHeader("Content-Type", "application/json");  
    res.end(JSON.stringify(usuarios));  
  
}
```

POST – Entendendo o body

- Dados enviados pelo cliente
- Não vêm na URL
- Chegam em partes (stream)
- Precisamos montar o conteúdo
- Converter para JSON
- Depois processar

POST /usuarios

```
if (req.url === "/usuarios" && req.method === "POST") {  
  let body = "";  
  req.on("data", chunk => {  
    body += chunk;  
  });  
  
  req.on("end", () => {  
    const novoUsuario = JSON.parse(body);  
    usuarios.push(novoUsuario);  
    res.end("Usuário criado");  
  });  
}
```

PUT – Atualização de dados

- Recebe ID pela URL
- Recebe novos dados no body
- Busca usuário existente
- Valida existência
- Atualiza propriedades
- Retorna resposta adequada

PUT /usuarios/:id

```
if (req.url.startsWith("/usuarios/") && req.method === "PUT") {  
  
  let body = "";  
  req.on("data", chunk => {  
    body += chunk;  
  });  
  
  req.on("end", () => {  
    const dados = JSON.parse(body);  
    const id = parseInt(req.url.split("/")[2]);  
    const usuario = usuarios.find(u => u.id === id);  
  
    if (!usuario) {  
      res.statusCode = 404;  
      res.end("Usuário não encontrado");  
      return;  
    }  
  
    usuario.nome = dados.nome;  
    res.end("Usuário atualizado");  
  });  
}
```

DELETE – Remoção de dados

- Recebe ID pela URL
- Localiza posição no array
- Remove usando splice
- Valida existência do usuário
- Evita erro silencioso
- Finaliza ciclo CRUD

DELETE /usuarios/:id

```
if (req.url.startsWith("/usuarios/") && req.method === "DELETE") {
```

```
    const id = parseInt(req.url.split("/")[2]);
```

```
    const index = usuarios.findIndex(u => u.id === id);
```

```
    if (index === -1) {
```

```
        res.statusCode = 404;
```

```
        res.end("Usuário não encontrado");
```

```
        return;
```

```
    }
```

```
    usuarios.splice(index, 1);
```

```
    res.end("Usuário removido");
```

```
}
```

Arquitetura em camadas (definição)

- Forma de organizar o código em responsabilidades
- Evita mistura de lógica
- Facilita manutenção
- Permite escalabilidade
- Muito usada no mercado
- Base de sistemas profissionais

Camadas da aplicação

- Controller → recebe e responde requisição
- Service → aplica regras de negócio
- Repository → acessa dados
- Servidor → roteamento
- Cada camada tem função única
- Evita código confuso

Fluxo completo da aplicação

- Cliente envia requisição HTTP
- Servidor identifica rota
- Controller processa requisição
- Service executa lógica
- Repository acessa dados
- Resposta retorna ao cliente

Principais erros HTTP

- 404 → recurso não encontrado
- 400 → requisição inválida
- 500 → erro interno do servidor
- Devem ser usados corretamente: Padrão de mercado

Backend Completo Integrado

- Agora unimos GET, POST, PUT e DELETE em único servidor funcional
- Simula backend real
- Fluxo completo de requisição
- Base para APIs reais

Testando a API

- GET: navegador
- POST/PUT/DELETE: Postman, ThunderClient ou similar
- Enviar JSON no body
- Verificar respostas